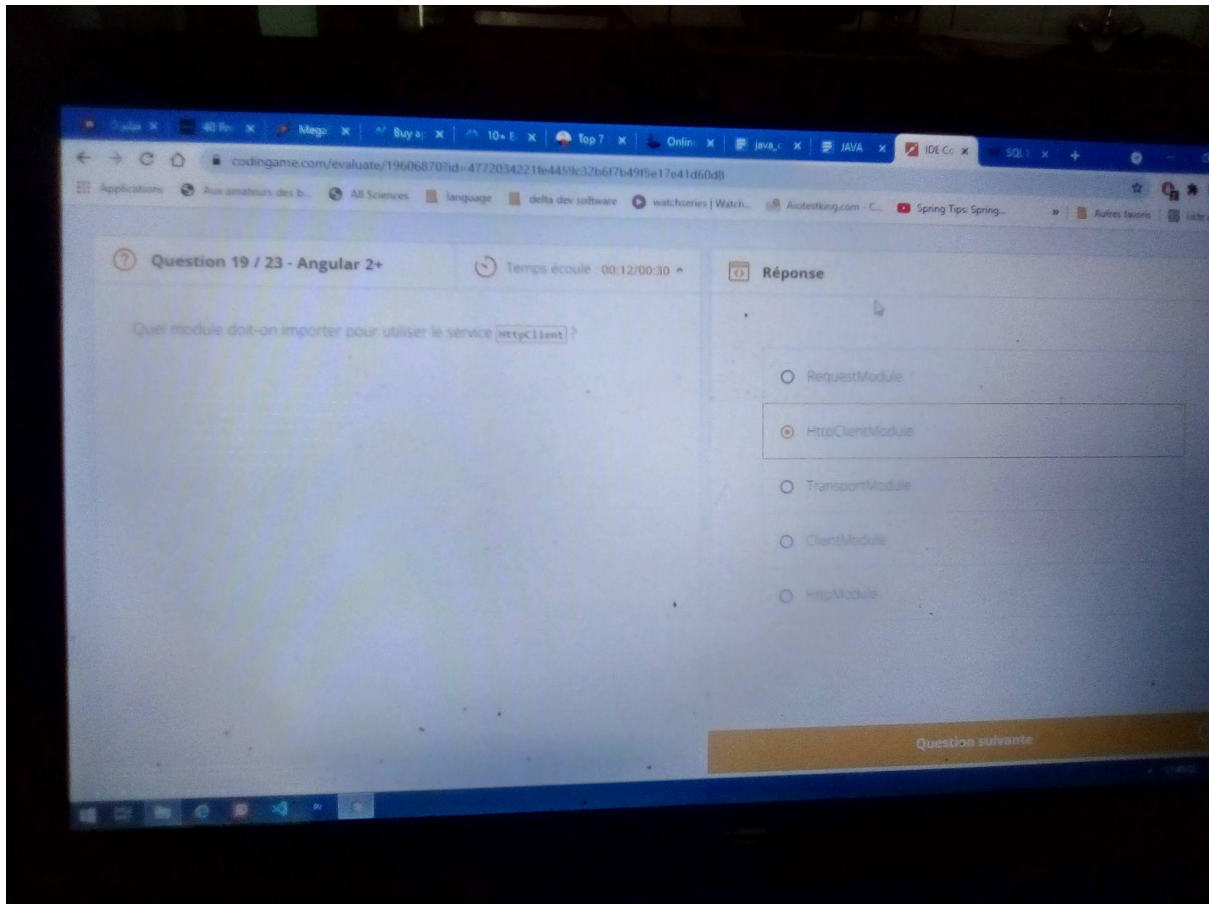


- Http client module

import { HttpClientModule } from '@angular/common/http';



- Angular new application

ng new my-angular-site

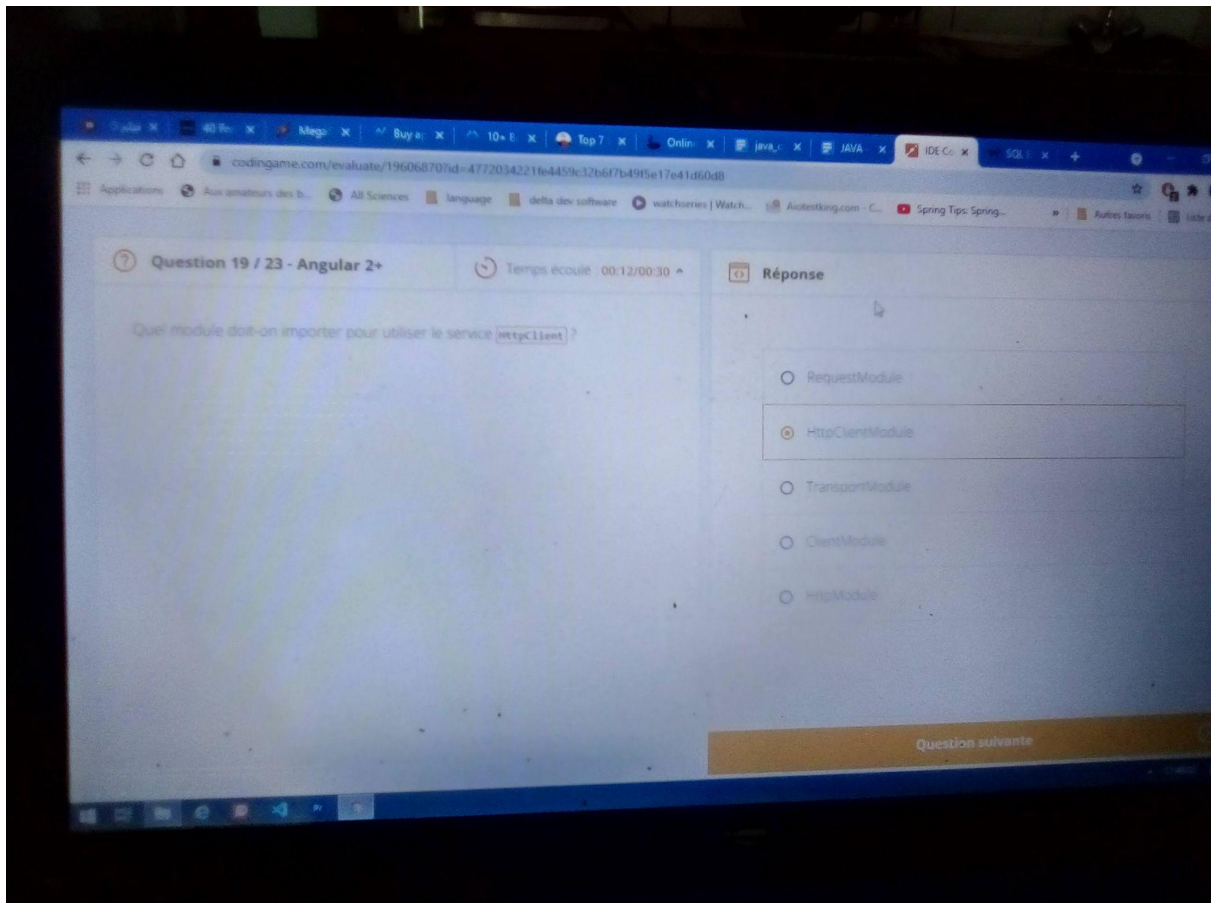
- Structural directive

They shape or reshape the DOM's structure, typically by adding, removing, or manipulating elements.

Exemple : NgIf, NgFor and NgSwitch

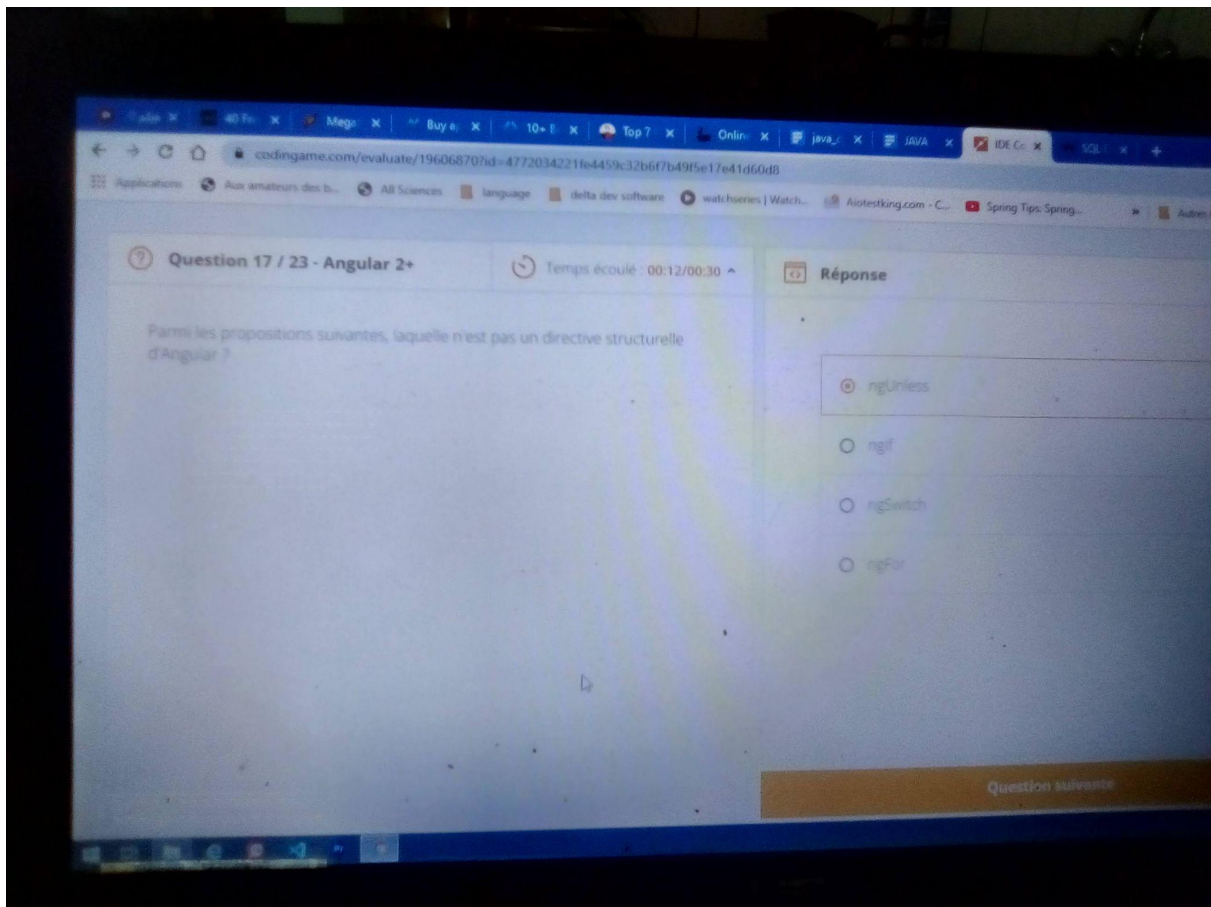
- String interpolation

. It uses the template expression in double curly braces **{{ }}** to display the data from component to view

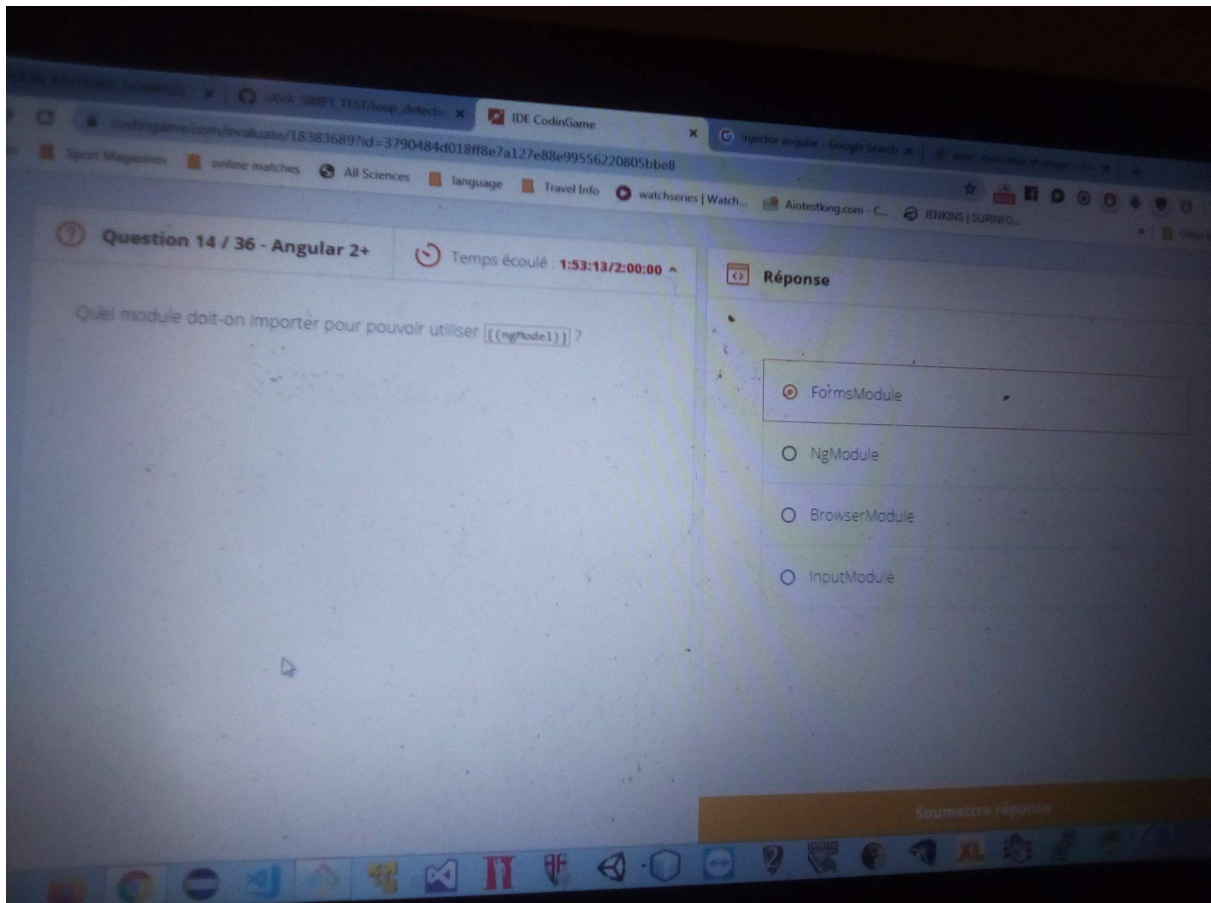


- Ng directive

-



- Ng model import module
`import { FormsModule } from '@angular/forms';`



- Angular cli

Angular CLI is a [command-line interface \(CLI\)](#) to automate your development workflow.

- Chrome debugging extension

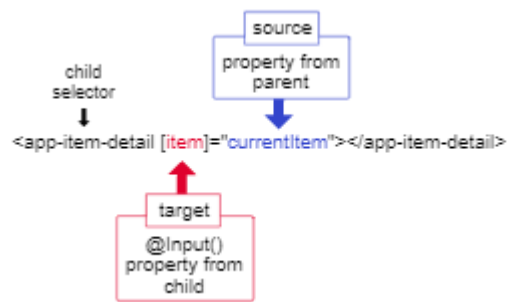
Augury

- Pipe (uppercase/lowercase/percent/currency/date)

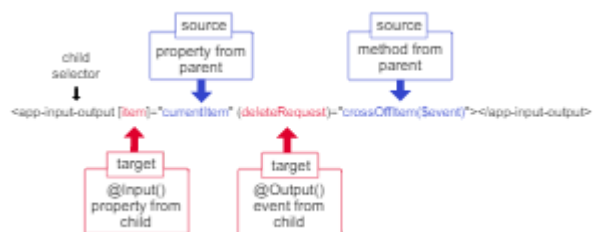
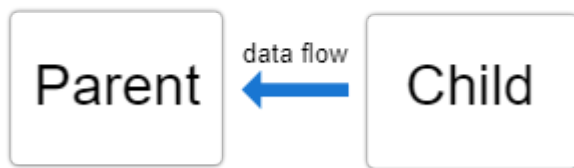
- input/output

@Input





@Output



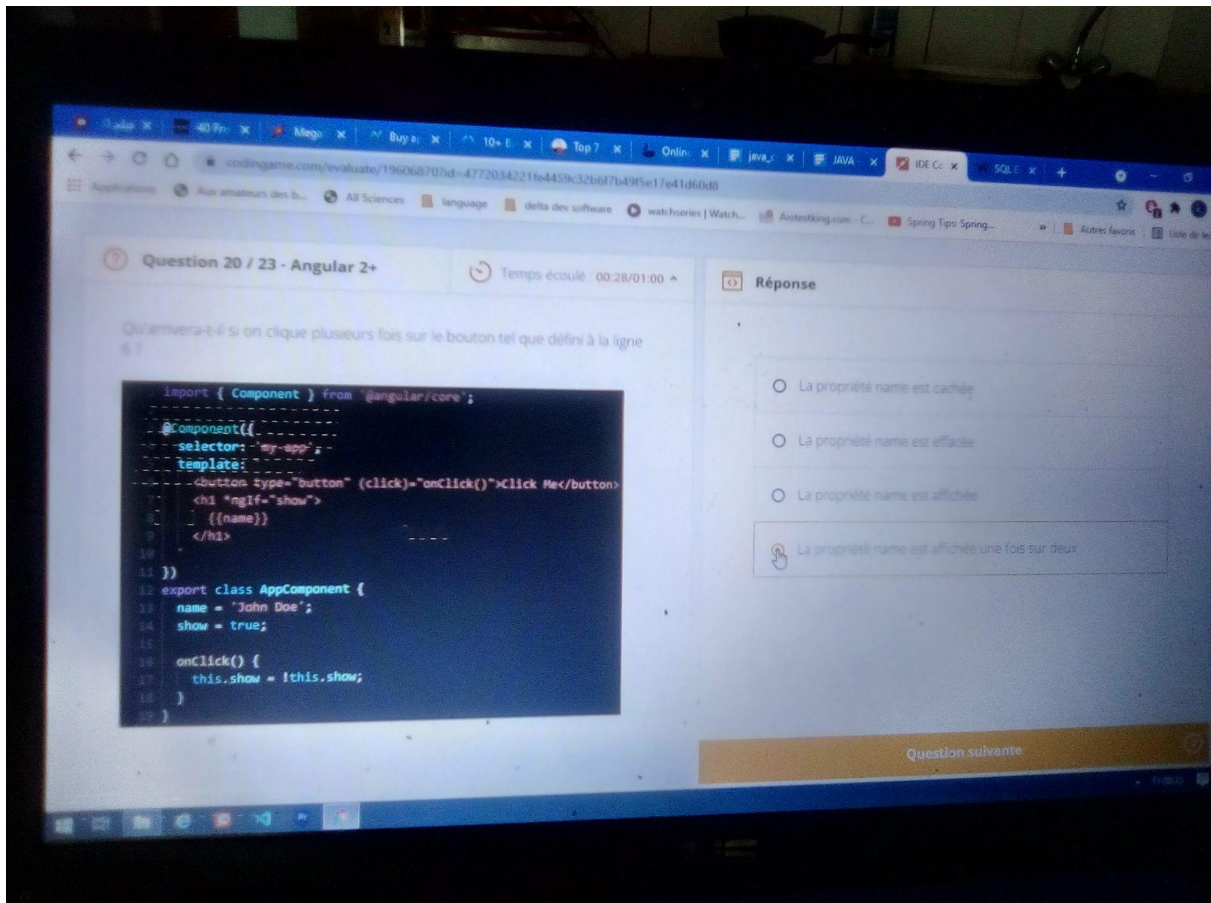
- Displaying a list

myData = { "id": "1", "name": "Name 1", "stuff": [{ "cmd": "something here" }, { "cmd": "something else here" }] }

view:

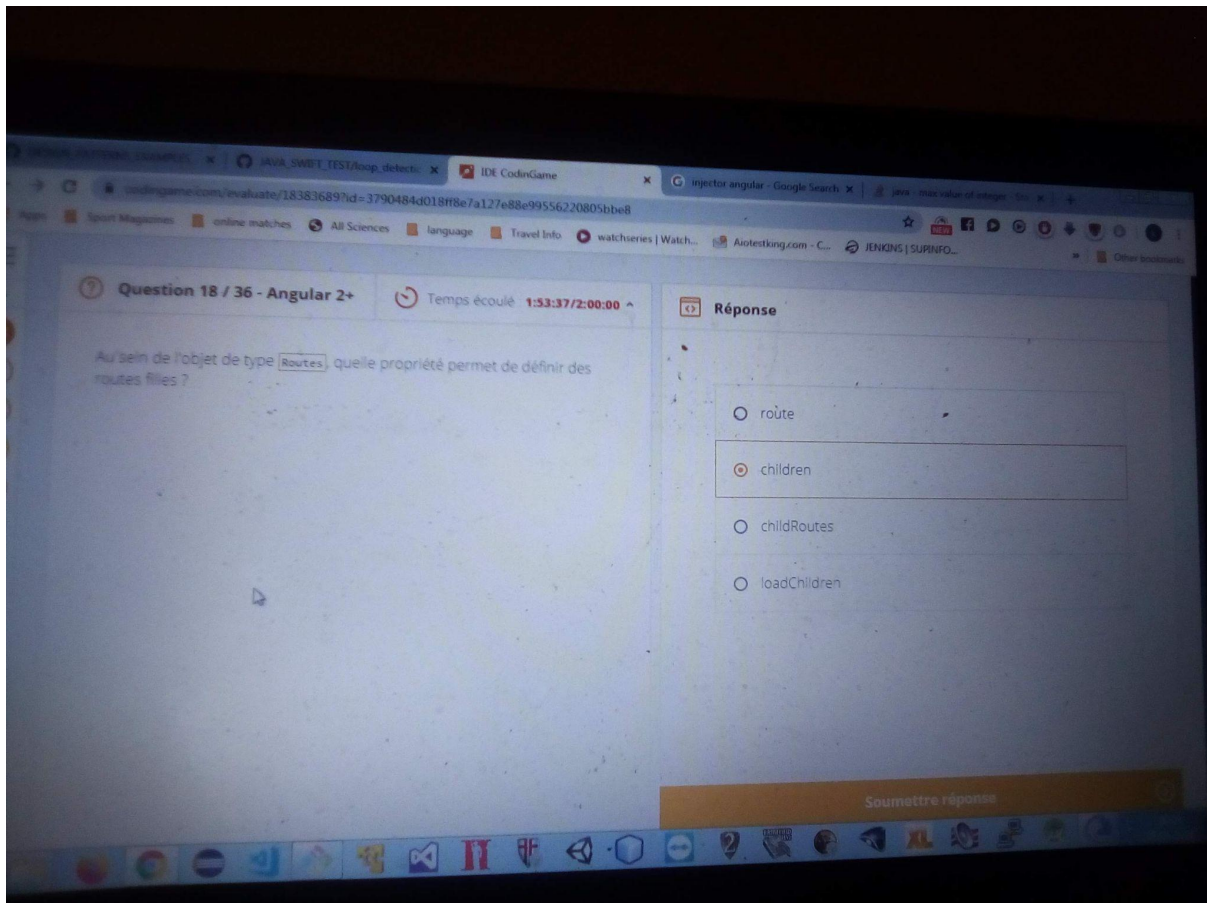
```
<ul class="code-editor-options-menu" *ngFor="let dat of myData.stuff"> <li>
```

```
<span>{{myData.name}}</span> <span aria-hidden="true">{{dat.cmd}}</span> </li> </ul>
```



Calling service : accounting test

- Defining child routes



```
const routes: Routes = [
  { path: '', component: ParentComponent, children: [
    { path: 'red', component: ChildComponent },
    { path: 'blue', component: AnotherChildComponent }
  ] }
];
```

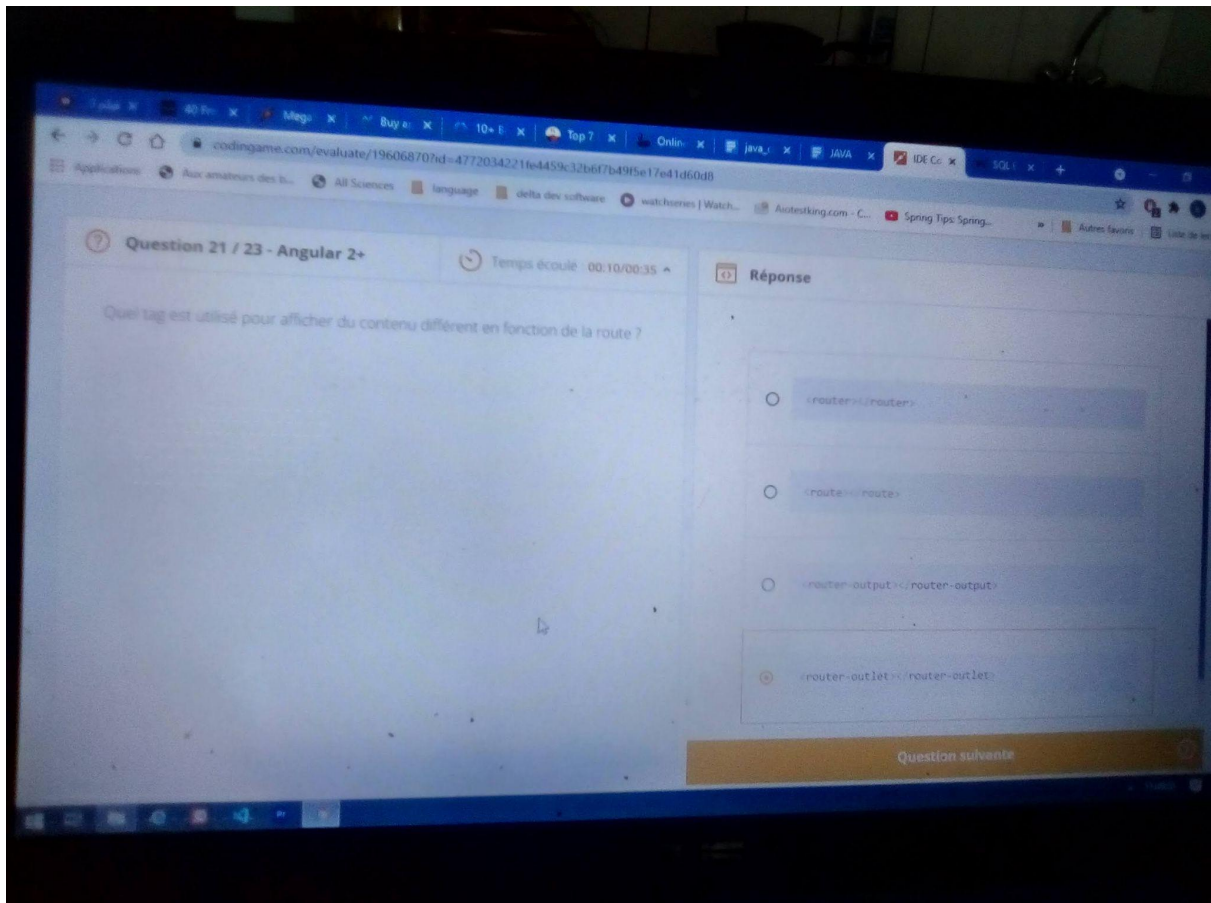
- Service to access router params

```
import { ActivatedRoute } from '@angular/router';
const queryParams =
this.activeRoute.snapshot.queryParams:observable
const routeParams = this.activeRoute.snapshot.params:observable;
```

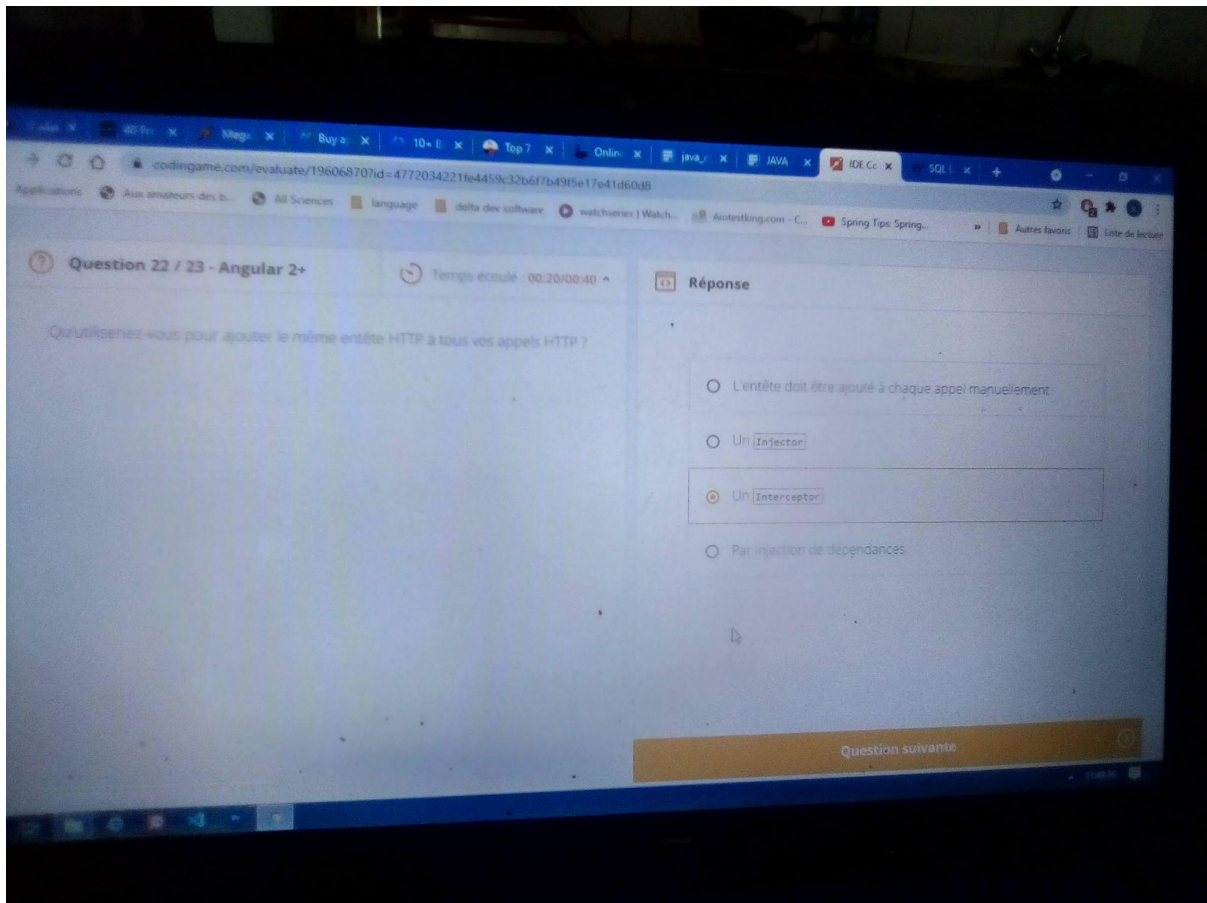
- Display of a component property

- Render routes contents

```
<router-outlet></router-outlet>
```



- Adding global http headers



Angular Interceptors technique comes in handy when you need to transform each request data, for instance, send an authorization token header with every HTTP request

Setting a header:

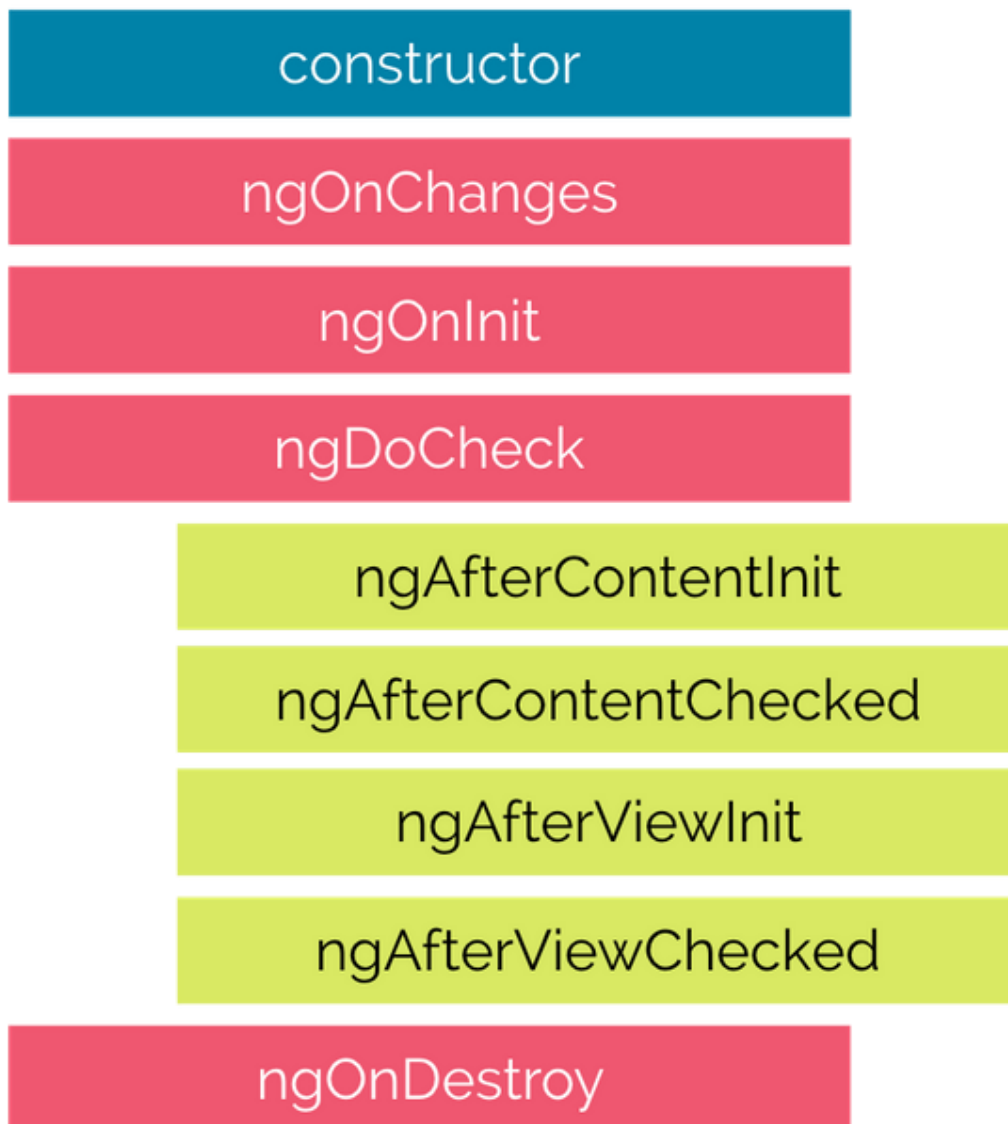
```
http .post('/api/items/add', body, { headers: new  
HttpHeaders().set('Authorization', 'my-auth-token'), }) .subscribe();
```

Setting headers:

```
this.http .post('api/items/add', body, { headers: new HttpHeaders({  
'Authorization': 'my-auth-token', 'x-header': 'x-value' }) }) .subscribe()
```

<https://medium.com/angular-shots/shot-3-how-to-add-http-headers-to-every-request-in-angular-fab3d10edc26>

- Angular life cycle hook



Listen to host events

```
@Component({
  selector: 'app',
  template: `<h1>Hello, you have pressed keys {{counter}} number of
times!</h1> Press any key to
increment the counter.
  <button (click)="resetCounter()">Reset Counter</button>`
})
class AppComponent {
  counter = 0;
  @HostListener('window:keydown', ['$event'])
```

```

handleKeyDown(event: KeyboardEvent) {
  this.counter++;
}
resetCounter() {
  this.counter = 0;
}
}

```

- Currency pipe

```

{{ 1 | currency:'USD' }}
<!--output 'CA$0.26'-->
<p>A: {{a | currency:'CAD'}}</p>

<!--output 'CAD0.26'-->
<p>A: {{a | currency:'CAD':'code'}}</p>

<!--output 'CA$0,001.35'-->
<p>B: {{b | currency:'CAD':'symbol':'4.2-2'}}</p>

```

- Angular directive types

Structural Directives : ngif, ngfor, ngswitch

Attribute Directives : ng style ng class

- Component creation

Ng generate component

- Routing basis

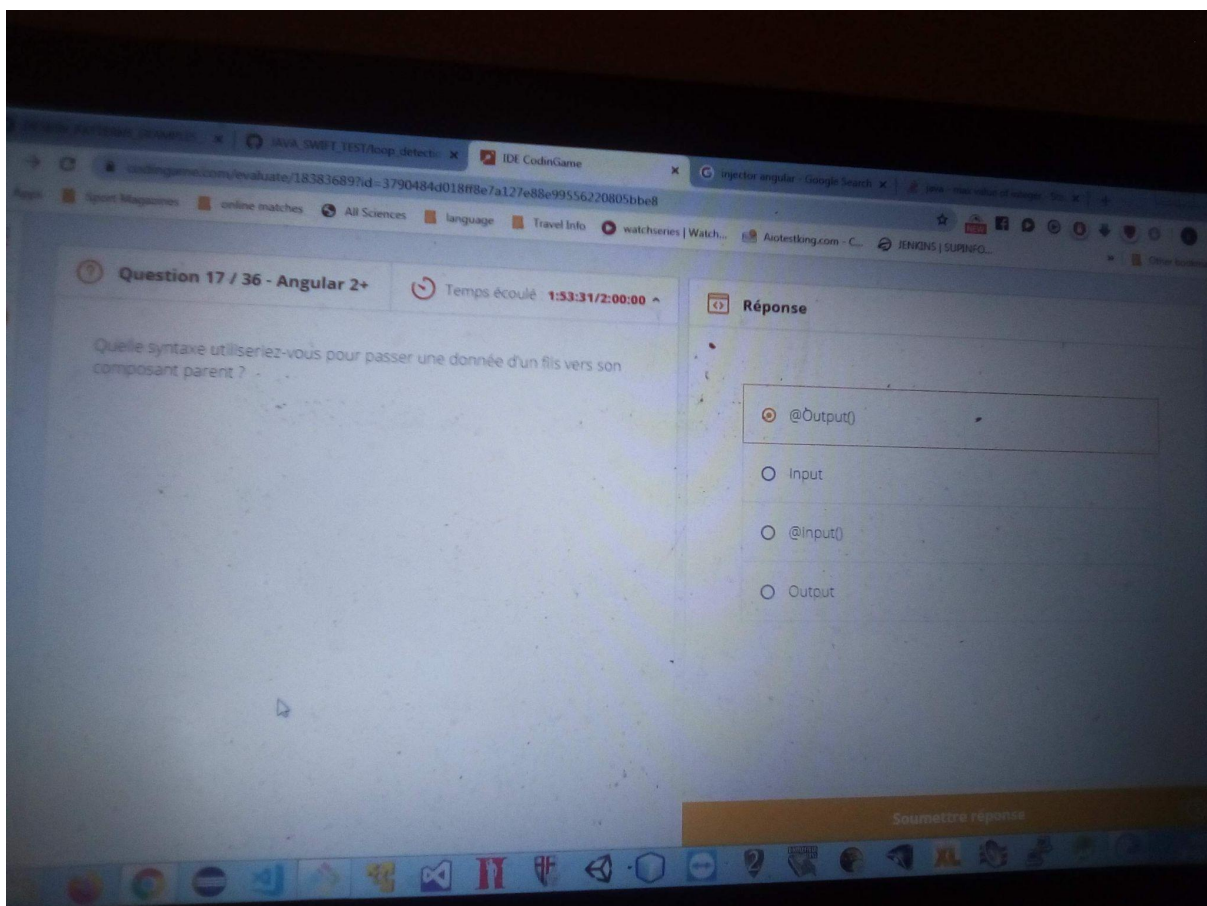
```

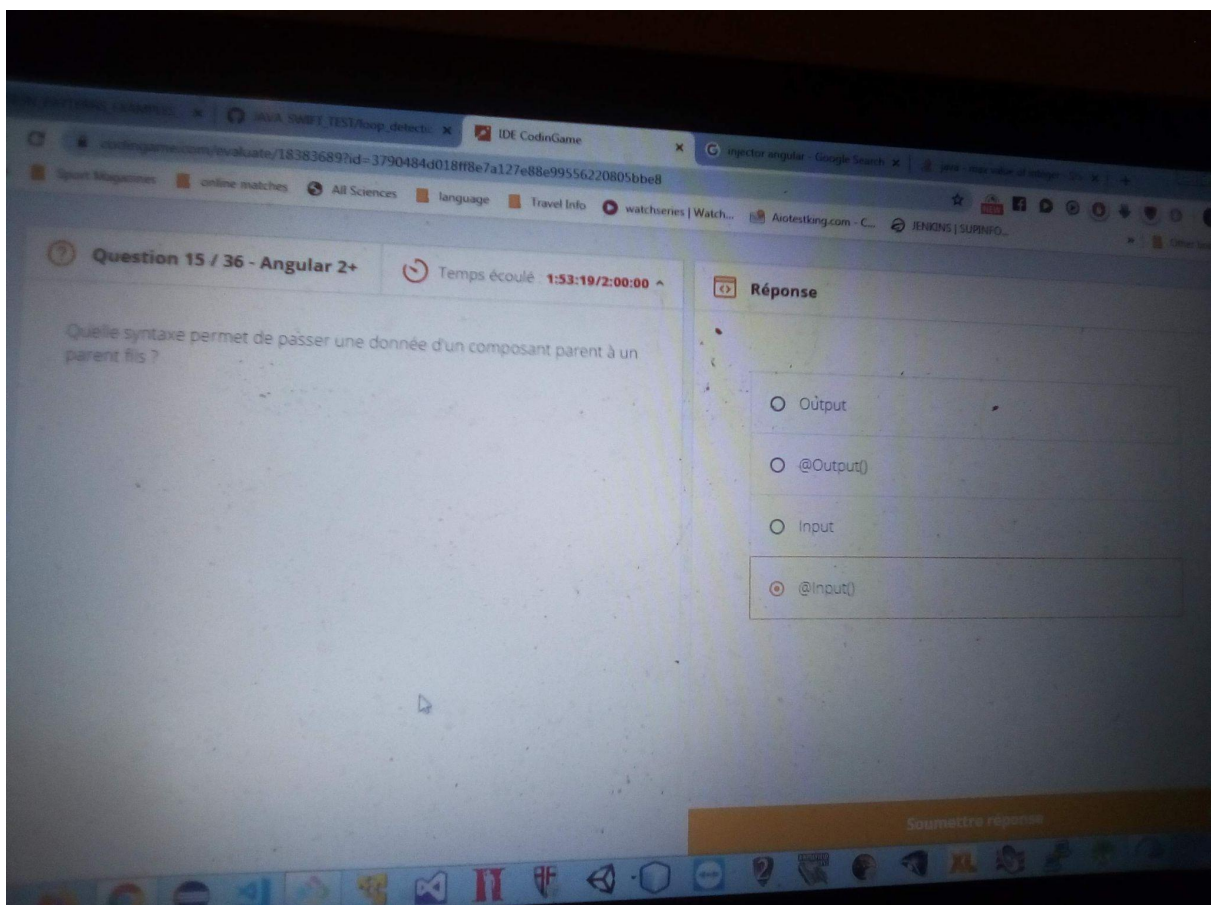
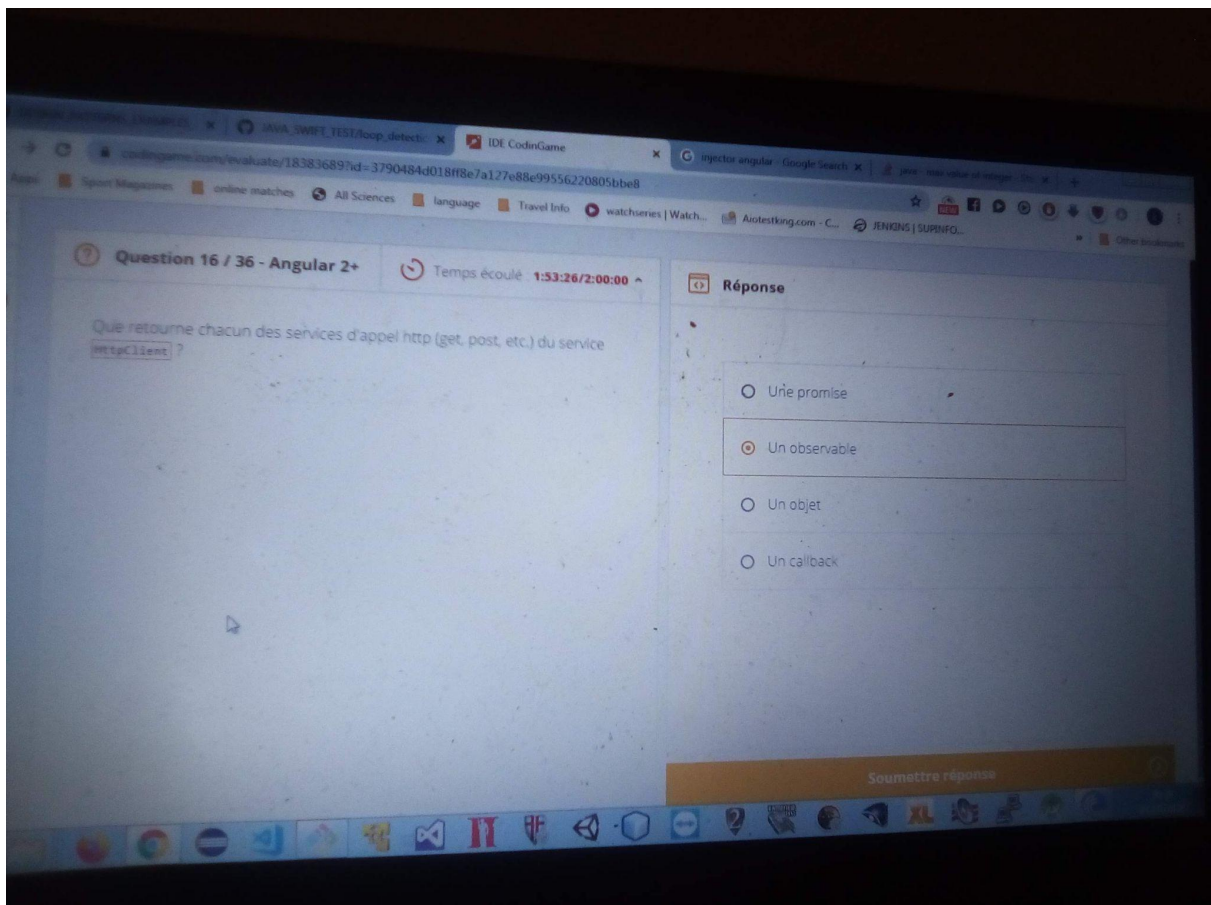
const routes: Routes = [
  {
    path:'balance', component: BalanceComponent
  }, {
    path:'deposit', component: DepositComponent
  }, {
    path:'withdraw', component:WithdrawComponent
  }
]

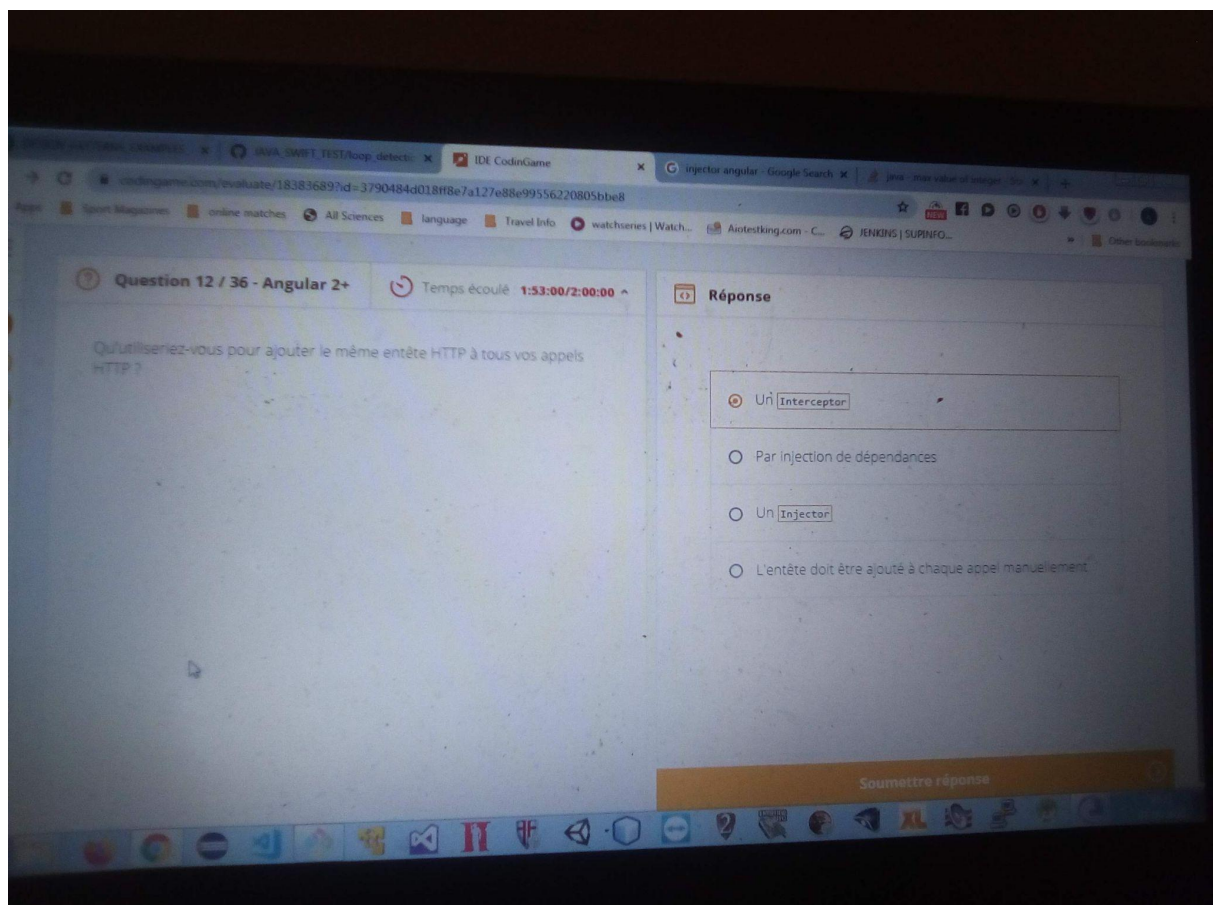
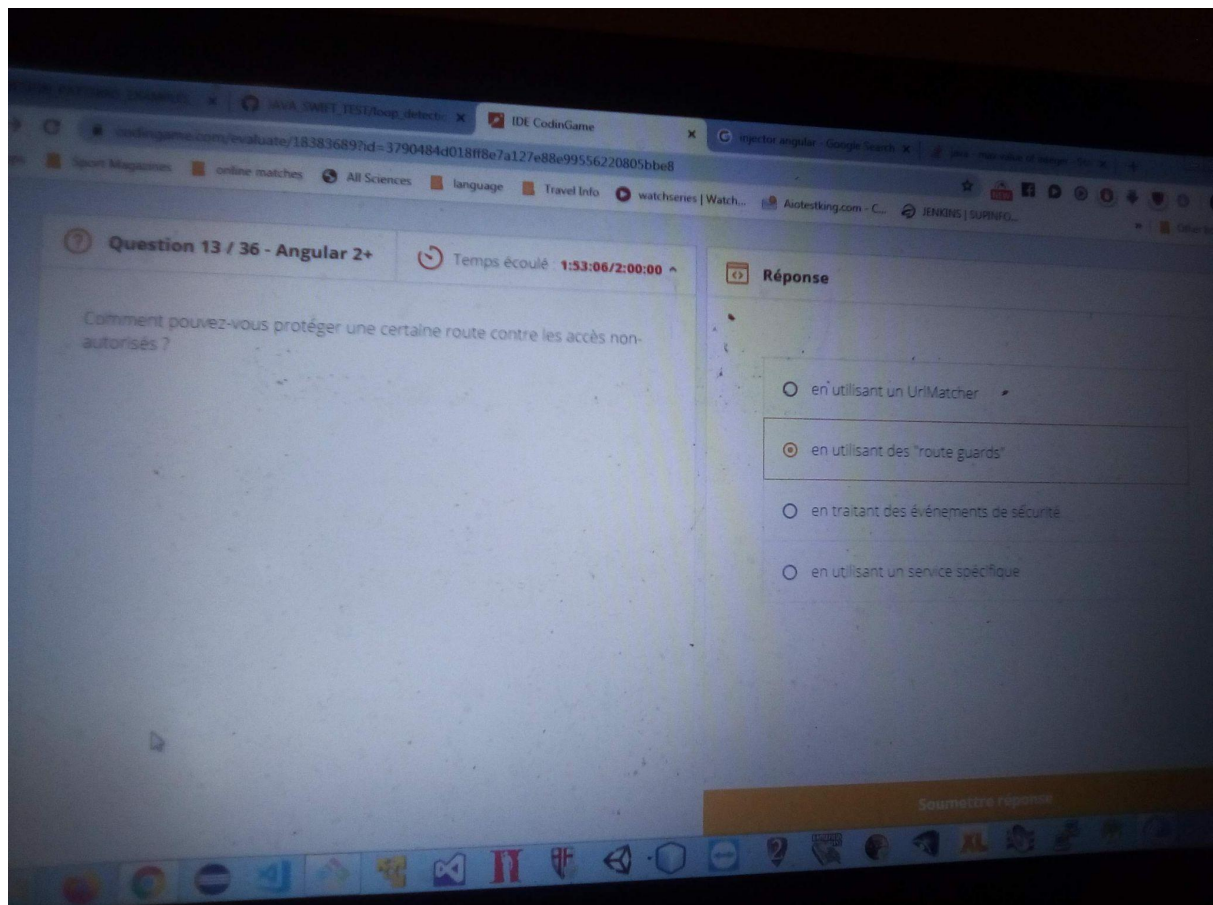
```

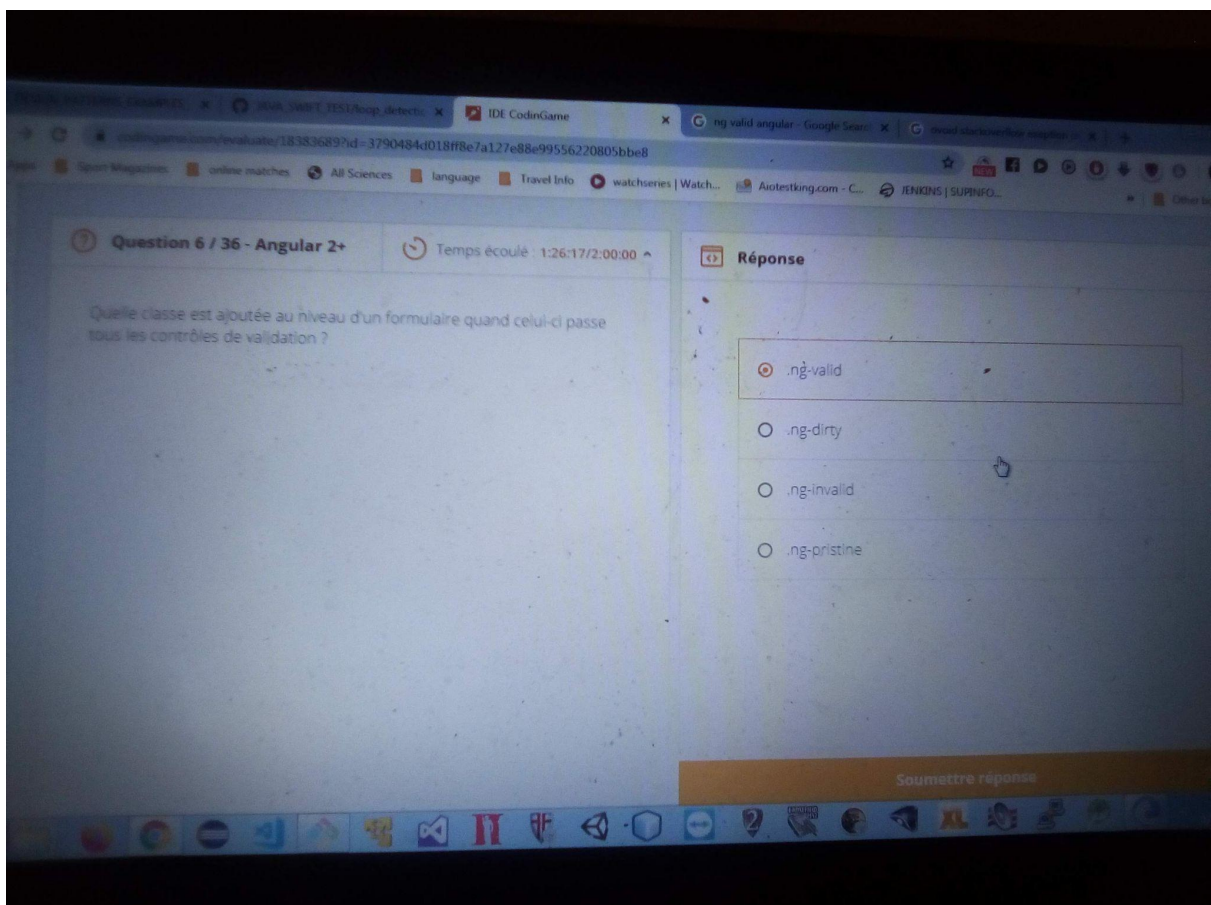
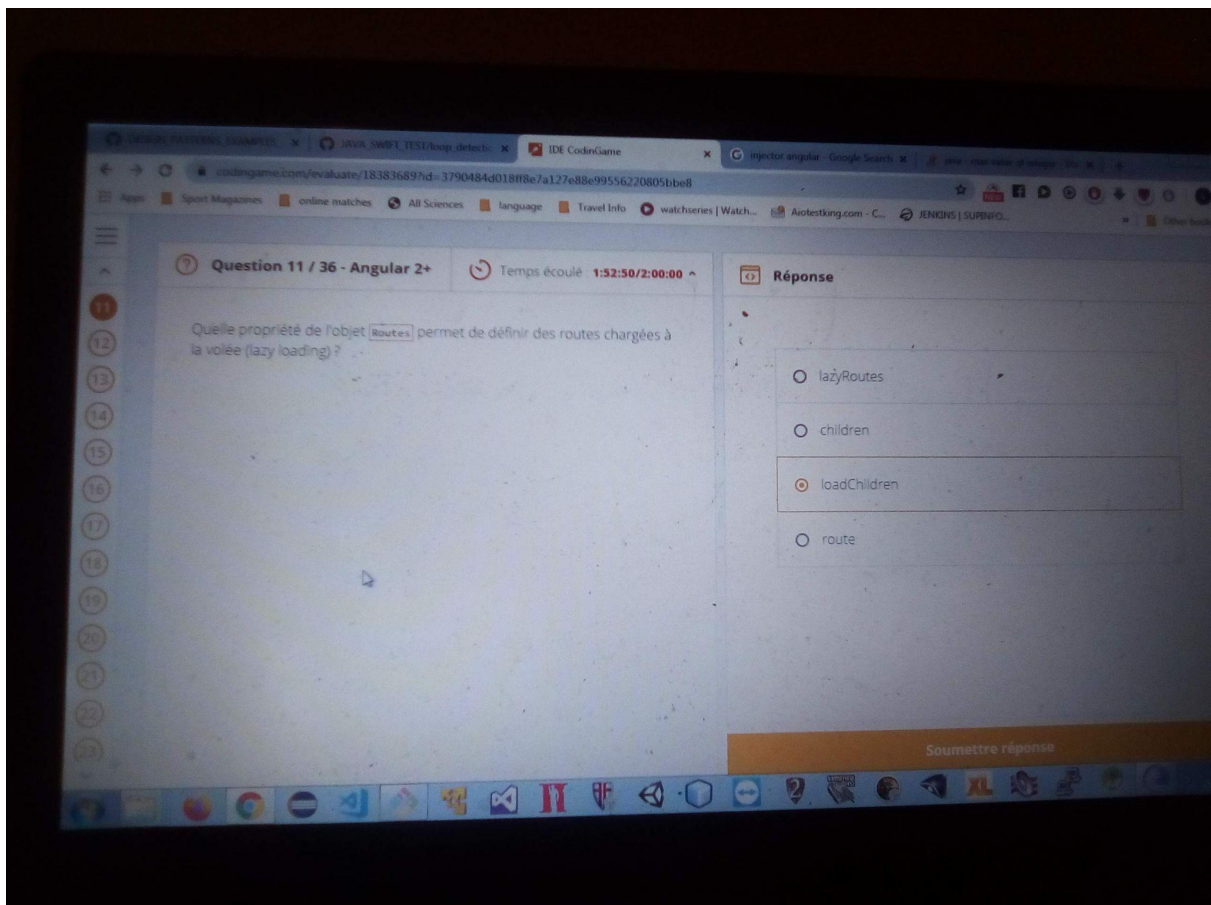
```
];
@NgModule ({
  imports: [RouterModule.forRoot(routes)],
  exports: [RouterModule]
})
```

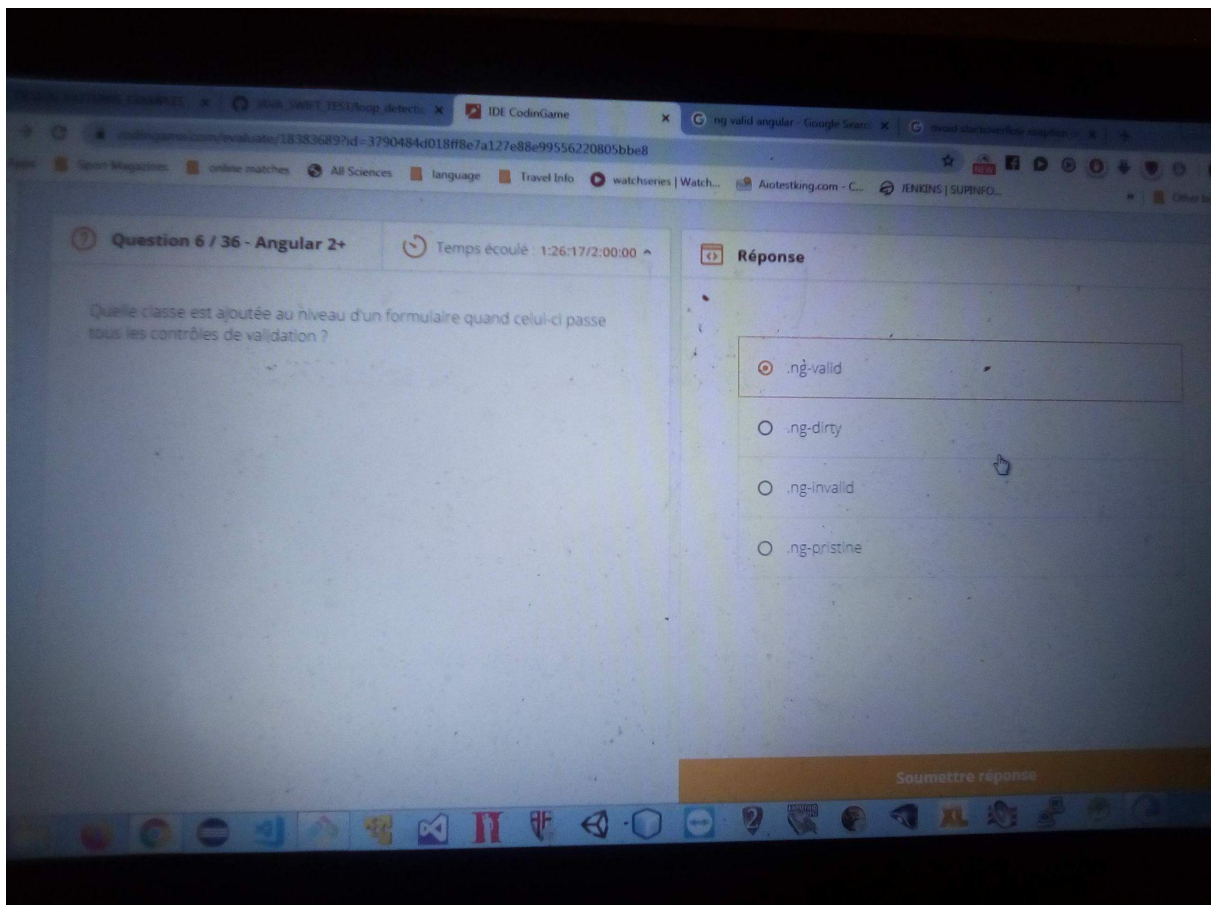
- `ng-pristine` No fields has not been modified yet
- `ng-dirty` One or more fields has been modified
- `ng-valid` The form content is valid
- `ng-invalid` The form content is not valid

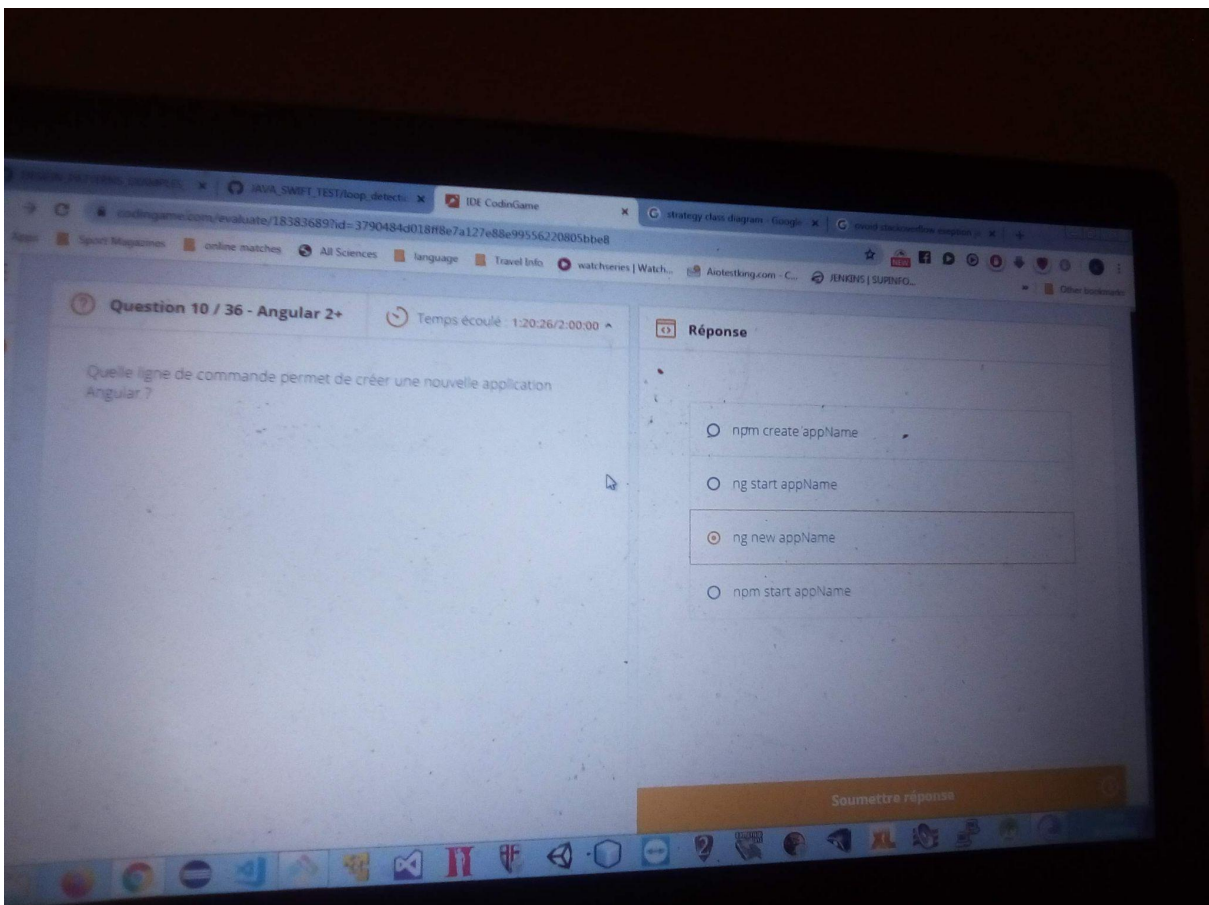
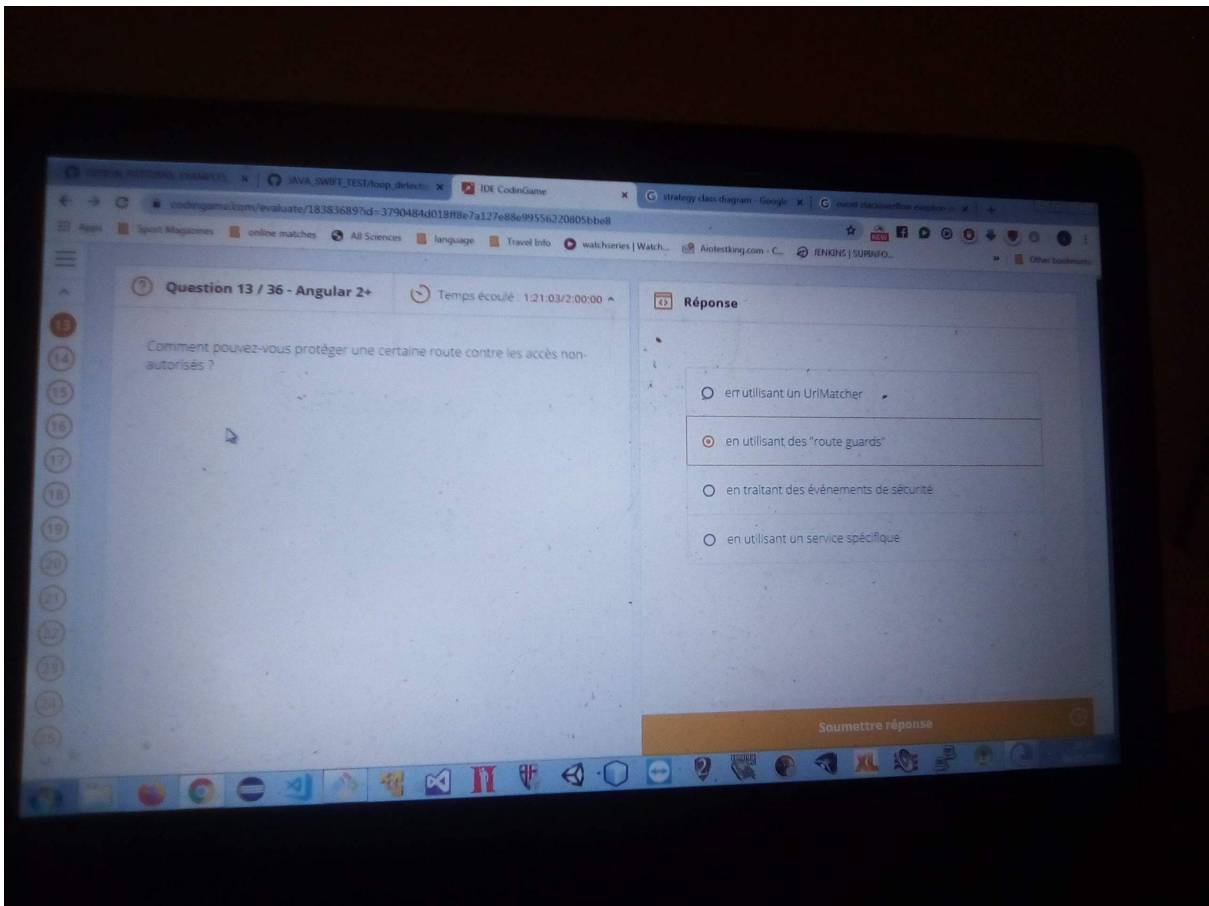


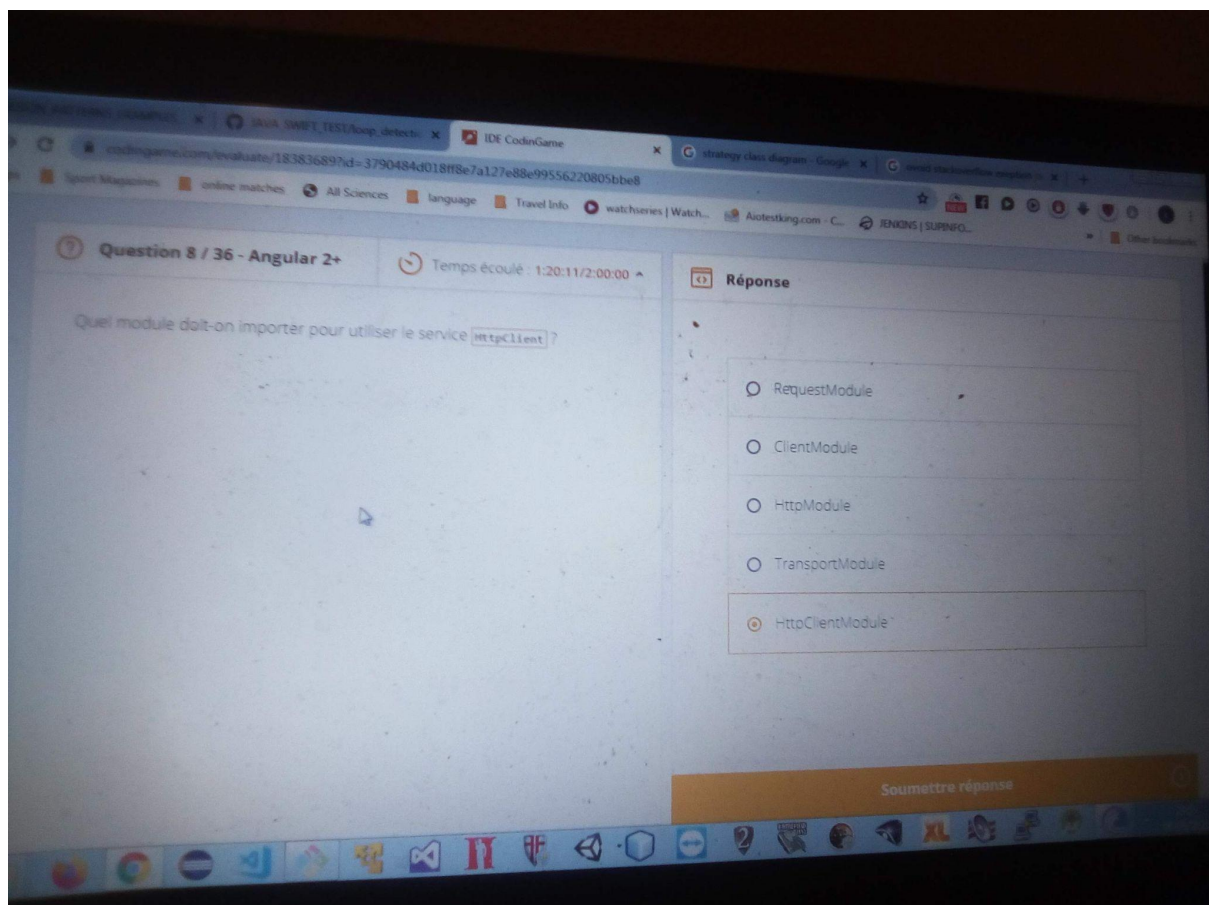
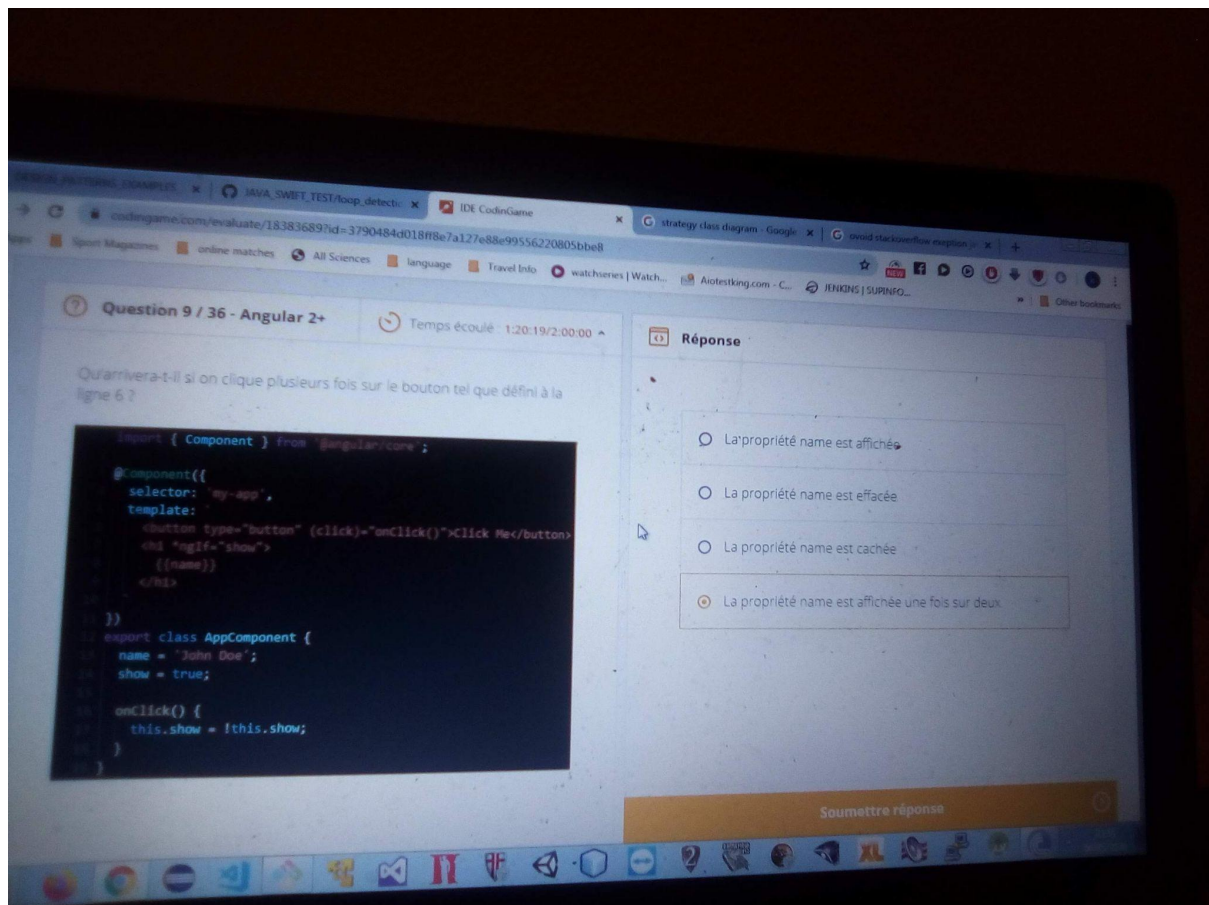


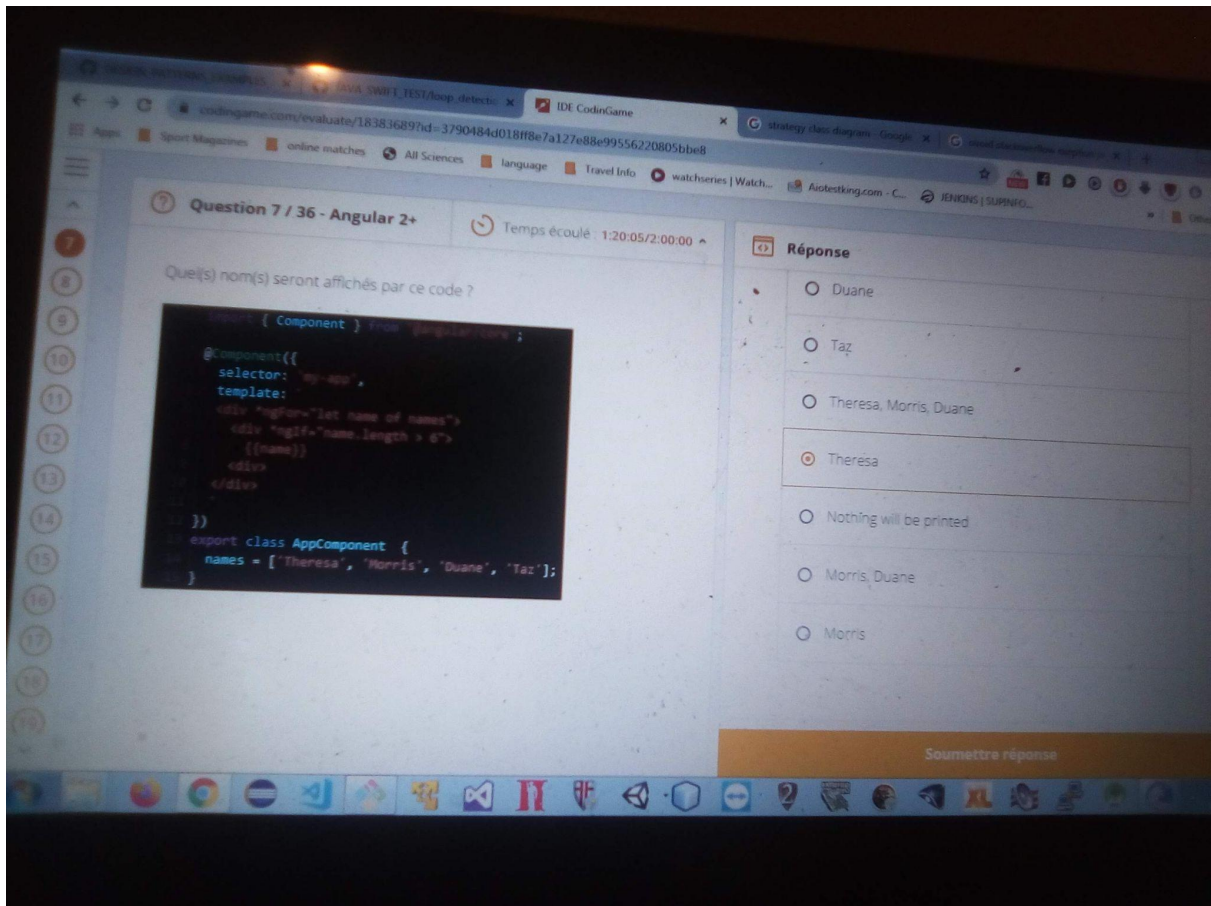


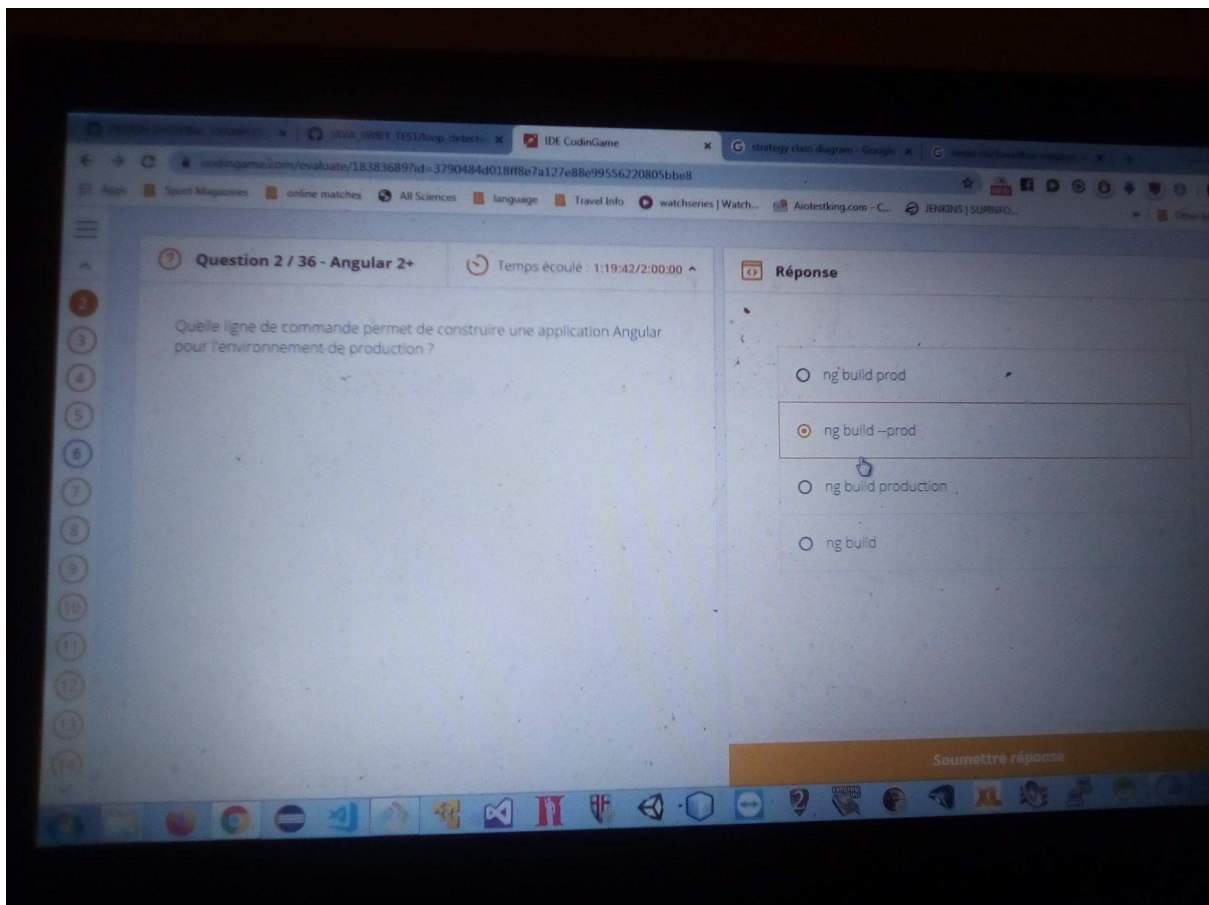
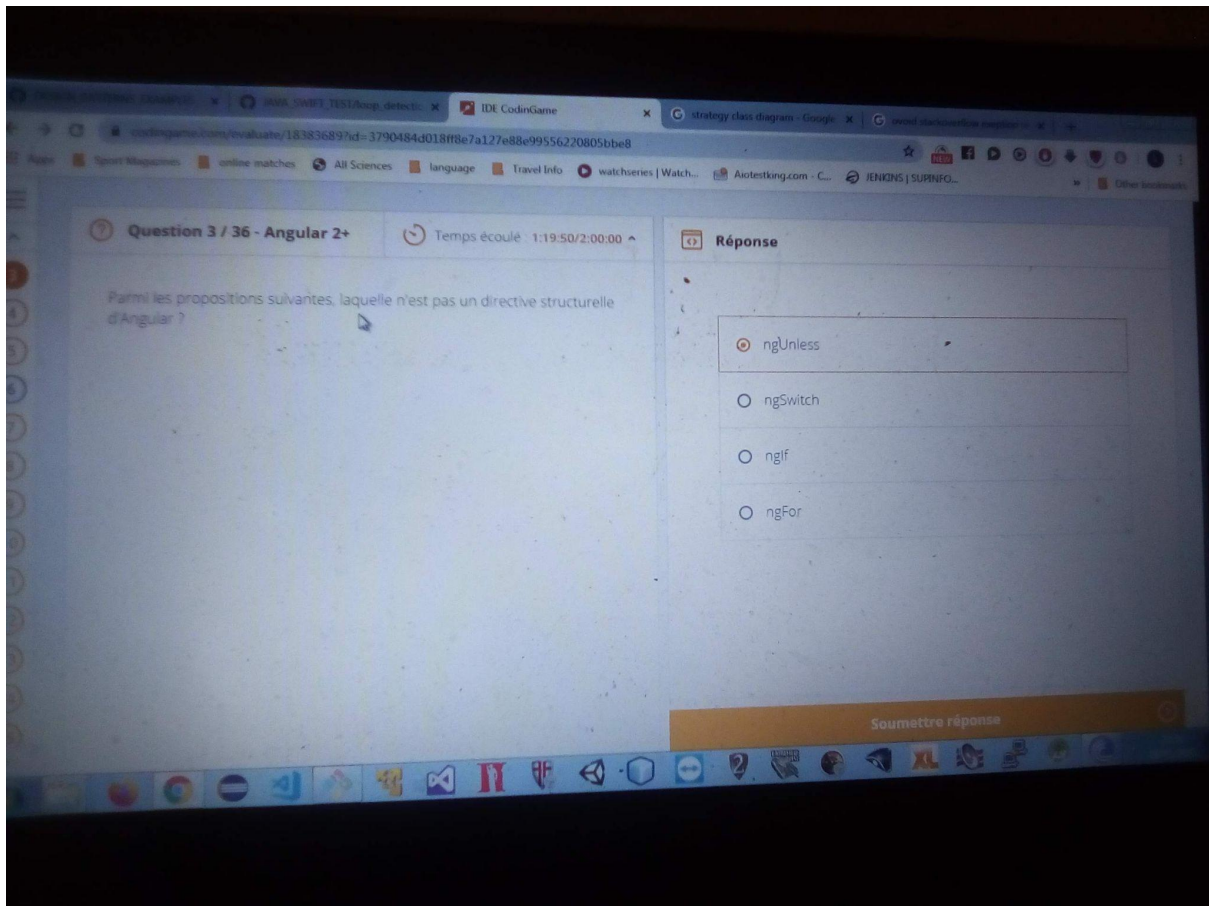


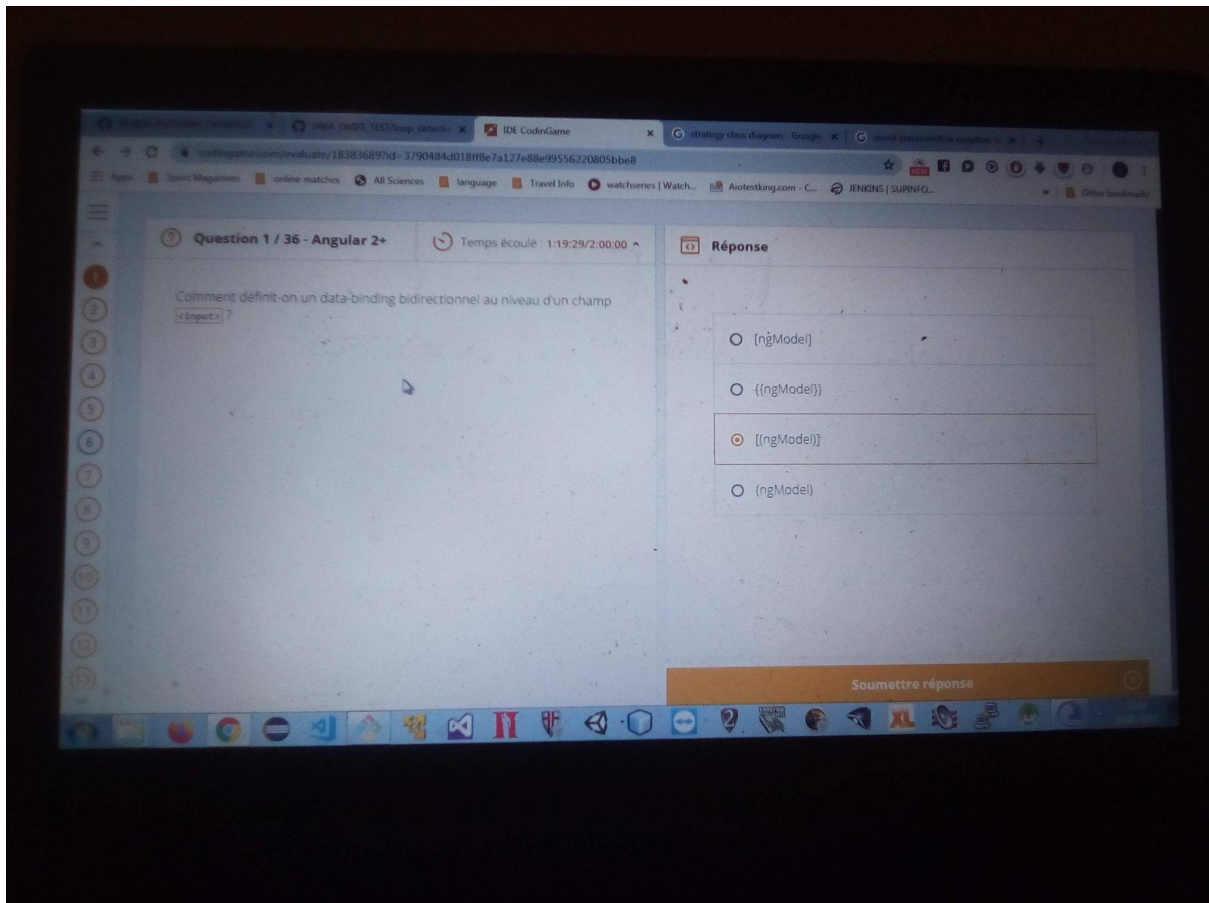












```
import { Component, Input, OnInit, Output } from '@angular/core';
import { interval, Observable, Subject } from 'rxjs';
import { filter, map, switchMap } from 'rxjs/operators';

@Component({
  selector: 'app-hello',
  templateUrl: './hello.component.html',
  styleUrls: ['./hello.component.css'],
})
export class HelloComponent implements OnInit {
  constructor() {}

  @Input() message: string;
  public inputValue: string = '';
  public buttonClicked: Subject<void> = new Subject();
  @Output() tick: Observable<string> = this.buttonClicked.pipe(
```

```

        map(() => parseInt(this.inputValue, 10)),
        filter((value) => Number.isInteger(value)),
        switchMap((value) => interval(value)),
        map(() => this.message)
    );
    ngOnInit(): void {}
}

<input type="number" [(ngModel)]="inputValue" /><button
    (click)="buttonClicked.next()"
>
    Set interval
</button>

```

```

import { Component } from '@angular/core';

@Component({
    selector: 'app-root',
    templateUrl: './app.component.html',
    styleUrls: ['./app.component.css'],
})
export class AppComponent {
    title = 'test-angular-input-output';
    public counter: number = 0;
    message: string = 'hello';

    public counterTick(message: string): void {
        this.message = message;
        this.counter++;
    }
}

<div>{{ message }} - {{ counter }}</div>
<app-hello [message]="message"

```



```
(tick)="counterTick($event)"></app-hello>
```